Schnorr-Based Implicit Certification: Improving the Security and Efficiency of Vehicular Communications

Paulo S. L. M. Barreto, Marcos A. Simplicio Jr, Jefferson E. Ricardini, and Harsh Kupwade Patil

Abstract—In the implicit certification model, the process of verifying the validity of the signer's public key is combined with the verification of the signature itself. When compared to traditional, explicit certificates, the main advantage of the implicit approach lies in the shorter public key validation data. This property is particularly important in resource-constrained scenarios where public key validation is performed very often, which is common in vehicular communications (V2X) that employ pseudonym certificates. In this article, we show an alternative Schnorr-based implicit certification procedure that can improve the efficiency of a popular V2X-oriented Vehicular Public Key Infrastructure (VPKI), the Security Credential Management System (SCMS). As an additional contribution, we show that SCMS's underlying certificate provisioning procedure, based on butterfly keys, is vulnerable to existential forgery attacks under certain conditions. We then discuss how this issue can be fixed in an effective and efficient manner.

Index Terms—Vehicular communications (V2X), implicit certificates, butterfly key expansion, security

1 Introduction

Public key authentication is the cornerstone for any security system that relies on public/private key pairs for digital signature, key exchange and/or asymmetric encryption. Traditionally, this is accomplished by means of explicit certificates, digital documents that enclose the subject's public key and are signed by a trusted Certificate Authority (CA). This is the case of the worldwide Internet, which relies basically on X.509 certificates for verifying the authenticity of web domains.

Despite the widespread adoption of explicit certificates, there are some alternative models in the literature that target specific scenarios. Among them, implicit certificates [1], [2] are of special interest because, when compared to explicit ones, they reduce the amount of information required for authenticating public keys. This better efficiency makes implicit certificates appealing in resource-constrained scenarios, as the Internet of Things (IoT) [3]. In particular, this is the approach recommended in one of the main security architectures for protecting vehicular communications (also known as "vehicle-to-everything", or V2X): the Security Credential Management System (SCMS) [4], [5]. For example, by adopting the Elliptic Curve Qu-Vanstone (ECQV) implicit certification model, SCMS creates certificates as small as 117 bytes [6].

In this article, we show that a more efficient public key verification process can be obtained if SCMS adopts an alternative implicit certification scheme. The proposed solution builds upon the Optimal Mail Certificate (OMC) implicit certification scheme [7], which enables optimization opportunities not available in ECQV. The result is that, by integrating our proposal with SCMS's (unified) butterfly key expansion procedure [4], [5], we obtain efficiency gains on the vehicles' side. In addition, the proposed scheme addresses potential insecurity issues that might occur in this scenario. In particular, it prevents forgeries against OMC and similar designs [8], as well as a novel existential forgery attack that applies specifically to butterfly keys.

The rest of this document is organized as follows. Section 2 briefly describes the goals and characteristics of ECQV implicit certificates. Section 3 introduces the proposed OMC-based implicit certification scheme and discusses its security and performance. Section 4 shows how the proposed approach can be integrated with SCMS and the corresponding performance gains; it also discusses existential forgery attacks against SCMS and how to avert them in our proposal. Finally, Section 5 concludes the discussion.

1.1 Basic Notations and Definitions

Along this document, we write G to denote the generator of an elliptic curve group $\mathbb{G}$ of prime order q. All operations in this group are made $(\bmod q)$; hence, for conciseness, we omit this modular reduction when describing existing and the hereby proposed protocols.

For convenience of the reader, Table 1 lists the main symbols employed in the cryptographic protocols hereby described. In addition to them, we write $v \xleftarrow{\$} \mathbb{Z}_q$ to denote that v is randomly sampled from $\mathbb{Z}_q$, the finite field of integers modulo q. We also denote by $a \stackrel{?}{=} b$ the procedure of verifying whether or not a and b have identical values.

2 Implicit Certification

The basic goal of digital certification is to bind public keys to their legitimate owners. In the traditional, explicit certification model, this is accomplished via a two-phase process. First, the user picks a private key u and computes by itself the corresponding public key $U \leftarrow u \cdot G$. Then, a trusted Certification Authority (CA) generates a digital certificate containing, at least, some system-defined metadata `meta`, the public key U, and the CA's signature on this data sig. A valid signature sig on certificate $cert : (\texttt{meta}, U, sig)$ usually implies that the owner of the certificate knows the private key u corresponding to U. Nonetheless, this knowledge can only be confirmed when U is actually used (e.g., by running a challenge-response protocol involving that key pair).

Conversely, in the implicit certification model the key pair (u, U) is computed in collaboration with the CA. More precisely, what is provisioned by the CA in this case is not a certificate, but a public key reconstruction credential V bound to that user. By using V together with the CA's public key C and the metadata `meta`, any entity can obtain the user's corresponding public key U. Even though the reconstructed U is not explicitly signed by the CA, its authenticity and its bond to a user can still be proved. This is done during key usage, whenever U's rightful owner shows that he/she knows the private counterpart u.

To give a concrete example, consider the main implicit certification solution in the literature, the Elliptic Curve Qu-Vanstone (ECQV) protocol [2]. ECQV's key provisioning procedure is shown in Table 2. First, the user computes a random elliptic curve (EC) private/public key pair $(x, X = x \cdot G)$, and then sends the public key X to the CA. The CA, in turn, picks a random integer r and computes a key reconstruction credential $V \leftarrow X + r \cdot G$. The CA then sets the user's implicit certificate to $cert \leftarrow \{V, \texttt{meta}\}$ and signs the result, obtaining $sig \leftarrow h \cdot r + c$ for $h \leftarrow \mathcal{H}(cert)$. Both $cert$ and sig are then sent back to the requesting user. Finally, the user computes its own private key as $u \leftarrow h \cdot x + sig$, and the corresponding public key as $U \leftarrow u \cdot G$.

- *Paulo S. L. M. Barreto is with the Institute of Technology, University of Washington Tacoma, WA 98402. E-mail: pbarreto@uw.edu.*
- *Marcos A. Simplicio Jr is with the Department of Computer and Digital Systems Engineering, University of Sao Paulo, Brazil. E-mail: mjunior@larc.usp.br.*
- *Jefferson E. Ricardini and Harsh Kupwade Patil are with LG Electronics, Santa Clara, CA 95054. E-mail: {jefferson1.ricardini, harsh.patil}@lge.com.*

Manuscript received 21 Apr. 2019; revised 22 Mar. 2020; accepted 11 Apr. 2020. Date of publication 20 Apr. 2020; date of current version 10 Feb. 2021.
(Corresponding author: Marcos A. Simplicio Jr.)
Recommended for acceptance by T. Güneysu.
Digital Object Identifier no. 10.1109/TC.2020.2988637

0018-9340 © 2020 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.
Authorized licensed use limited to: Duke University. Downloaded on February 17,2026 at 16:37:47 UTC from IEEE Xplore. Restrictions apply.

TABLE 1
General Notation and Symbols

Symbol	Meaning
sig	A digital signature
$cert$	A digital certificate
$\texttt{meta}$	A digital certificate's metadata
U, C	Public signature keys (C: reserved for PCA)
u, c	Private keys associated to U and C (respectively)
X, x	Public and private caterpillar keys
$\hat{X}, \hat{x}$	Public and private cocoon keys
V	Public key reconstruction credential (implicit model)
β	Number of cocoon keys in a batch of certificates
f	A pseudorandom function (PRF)
$\mathcal{H}(str)$	Hash of bitstring str
$Enc(\mathcal{K}, str)$	Encryption of bitstring str with key $\mathcal{K}$

In possession of $cert$, any user can reconstruct the public key U by computing $h \leftarrow \mathcal{H}(cert)$, and check its authenticity by verifying the equation $U \stackrel{?}{=} h \cdot V + C$. This equation is valid because $U = u \cdot G = (h \cdot x + sig) \cdot G = (h \cdot x + h \cdot r + c) \cdot G = h \cdot (x \cdot G + r \cdot G) + c \cdot G = h \cdot V + C$. As discussed in [1], this procedure prevents forgery of a valid key reconstruction credential V by attackers that do not know the CA's private key c used for signing $cert$. The scheme is also escrowless, i.e., the CA does not learn the user's final private key u.

Commonly, the metadata enclosed in $cert$ includes some user-identifiable information [1], [2], similarly to the "subject" field in ordinary certificates. Nevertheless, by omitting such identity information, one can obtain anonymous (or, rather, pseudonym) implicit certificates. In that case, the bearer of the certificate, after proving knowledge of the corresponding private key, is considered authorized in the system despite not being identified.

3 More Efficient Implicit Certificates

The main performance shortcoming of the ECQV protocol shown in Section 2 lies in its credential verification equation, $U \stackrel{?}{=} h \cdot V + C$. Specifically, the processing cost of this equation is dominated by the EC-point multiplication $h \cdot V$, where the V point is expected to be different for each user. Therefore, this operation is not amenable to optimization methods typical of fixed-point EC multiplications (see [9] [Section 3.3.2] for a few examples). In other words, for a generic choice of the underlying elliptic curve, this procedure cannot be accelerated by pre-computing and storing data that depend only on the point V.

This issue can be addressed with a slightly different implicit certification protocol, which takes full advantage of fixed-point multiplications. The proposed solution is shown in Table 3, in which we highlight (in gray background) the differences toward ECQV. The first distinction in this proposal is that, like the Optimal Mail Certificate (OMC) [7], we employ the elliptic curve version of the Schnorr signature algorithm [10] when computing $sig = r + h \cdot c$. The private key reconstruction operation then becomes $u \leftarrow x + sig$, whereas the public key reconstruction algorithm is given by $U = V + h \cdot C$. The correctness of these equations can be verified by observing that $U = u \cdot G = (x + sig) \cdot G = (x + r + h \cdot c) \cdot G = (x \cdot G + r \cdot G) + h \cdot (c \cdot G) = V + h \cdot C$.

TABLE 2
ECQV Implicit Certificate Provisioning Protocol

usr	$\rightarrow$	CA	$\rightarrow$	usr
$x \stackrel{\$}{\leftarrow} \mathbb{Z}_q$ $X \leftarrow x \cdot G$	X	$r \stackrel{\$}{\leftarrow} \mathbb{Z}_q$ $V \leftarrow X + r \cdot G$ $cert \leftarrow \{V, \texttt{meta}\}$ $h \leftarrow \mathcal{H}(cert)$ $sig \leftarrow h \cdot r + c$	$cert, sig$	$h \leftarrow \mathcal{H}(cert)$ $u \leftarrow h \cdot x + sig$ $U \leftarrow u \cdot G$ $U \stackrel{?}{=} h \cdot V + C$

TABLE 3
Schnorr-Based Implicit Certificate (SIMPL) Provisioning Protocol

usr	$\rightarrow$	CA	$\rightarrow$	usr
$x \stackrel{\$}{\leftarrow} \mathbb{Z}_q$ $X \leftarrow x \cdot G$	X	$r \stackrel{\$}{\leftarrow} \mathbb{Z}_q$ $V \leftarrow X + r \cdot G$ $cert \leftarrow \{V, \texttt{meta}\}$ $h \leftarrow \mathcal{H}(cert, C)$ $sig \leftarrow r + h \cdot c$	$cert, sig$	$h \leftarrow \mathcal{H}(cert, C)$ $u \leftarrow x + sig$ $U \leftarrow u \cdot G$ $U \stackrel{?}{=} V + h \cdot C$

The second difference is that, similarly to [11], the intended verification public key is included in the hash computation by making $h = \mathcal{H}(cert, C)$. This approach is recommended for preventing certificate misbinding attacks [12], in which the wrong CA public key is associated with the implicit credential. In particular, this prevents the following attack against the system's consistency. Suppose that adversary $\mathcal{A}$ acts as an intermediary in the communication between the CA and the user usr requesting an implicit certificate. Suppose also that, at first, usr does not know the CA's public key C, which is expected to be provided together with the pair (V, sig) in the CA's response. In this scenario, if $h = \mathcal{H}(cert)$, the attacker could replace the CA's legitimate response (V, sig, C) with (V, sig', C'), where $sig' = sig + h \cdot z$ and $C' = C + z \cdot G$ for an arbitrary z. The user would then be able to compute its implicit keys and verify the correctness of the provided credential, since we would have $U = u \cdot G = (x + sig') \cdot G = (x + r + h \cdot c + h \cdot z) \cdot G = (x \cdot G + r \cdot G) + h \cdot (c \cdot G) + z \cdot G) = V + h \cdot (C + z \cdot G) = V + h \cdot C'$.

Hence, the attacker is able to convince usr to use the wrong CA public key even without knowing the corresponding private key c' (or, equivalently, the legitimate $c = c' - z$). Such inconsistency issue may not be critical in some contexts, in particular when users know the CA's public key beforehand. Nevertheless, ensuring consistency by design is at least advisable. The reason is that this avoids subtle attacks that might be built against different systems adopting the proposed implicit certification scheme.

An effective approach to address this matter is via "key prefixing" [13]: when some piece of data M needs to be signed, the signature algorithm's input includes both M and some information derived from the signer's public key. For example, when the signer uses explicit certificates, its public key can be used directly [14]; this is the case assumed in the SIMPL protocol, where $(cert, C)$ is signed instead of $M = cert$. For implicit certificates, though, the message M might be combined with the certificate $cert$ itself [4] or with that certificate's hash [11]. The benefit in this case is that the verification of a signature over $(M, cert)$ does not require the prior computation of the signer's public key U, since any operation involving its value can be computed using $V + h \cdot C$ instead. Nonetheless, in scenarios where many signatures are made under the same U, the cost of computing U can be compensated by a faster verification of such signatures (this is further discussed in Section 4.3.2). Therefore, and even though we do not impose any specific approach for which information derived from U should be used in key prefixing, in our notation we use U directly for the sake of simplicity.

Despite the underlying details, the processing cost of the key prefixing technique is expected to be very low. After all, it translates simply to a small increase in the amount of data fed to the hash function. At the same time, this approach is highly effective against misbinding attacks. In SIMPL itself, as long as $h = \mathcal{H}(cert, C) \neq \mathcal{H}(cert, C')$, any attempt to replace C with C' in the CA's response would invalidate the CA's signature over h. In addition, as shown in Sections 3.3 and 4.2, enforcing this technique when signing data in the V2X scenario is useful to avoid forgery attacks that builds upon the properties of butterfly keys.

3.1 Performance Considerations

With this Schnorr-based implicit certification (SIMPL, for short) approach, the implicit certificate's hash h is multiplied by the CA's

Authorized licensed use limited to: Duke University. Downloaded on February 17,2026 at 16:37:47 UTC from IEEE Xplore. Restrictions apply.

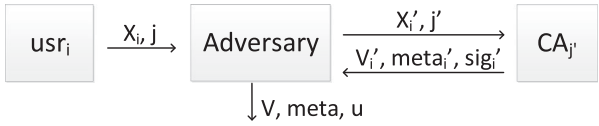


Fig. 1. Security model for implicit certificates. Adapted from [1].

long-term private key c, instead of the ephemeral private key r. As a result, the public key reconstruction algorithm $U \stackrel{?}{=} V + h \cdot C$ requires one EC multiplication by what is commonly a fixed point: the CA's public key C. This operation can, thus, take full advantage of optimizations such as pre-computation or dedicated code. For instance, suppose that h is 256-bits long and is written in hexadecimal, so $h = h_{63} \cdot 16^{63} + h_{62} \cdot 16^{62} + ... + h_1 \cdot 16 + h_0$ with $0 \leqslant h_p < 16$ for all $0 \leqslant p < 64$. Then, by pre-computing and storing a single, fixed 64×16 table $T[p][d] := (d \cdot 16^p) \cdot C$, the $h \cdot C$ operation can be implemented via table look-ups as follows:

$$\begin{aligned} h{\cdot}C &= (h_{63}{\cdot}16^{63} + h_{62}{\cdot}16^{62} + \ldots + h_1{\cdot}16 + h_0){\cdot}C \\ &= (h_{63}{\cdot}16^{63}){\cdot}C + (h_{62}{\cdot}16^{62}){\cdot}C + \ldots + (h_1{\cdot}16){\cdot}C + h_0{\cdot}C \\ &= T[63][h_{63}] + T[62][h_{62}] + \ldots + T[1][h_1] + T[0][h_0] \end{aligned}$$

Ignoring the (usually small) cost of table look-ups, this approach would take only 63 point additions. In comparison, a regular double-and-add method would require 255 point doublings, plus 128 additions on average (or roughly 85 additions with a Non-Adjacent Form representation). Thus, this table-based technique is expected to be about 3 to 4 times faster at the cost of storing a fixed 32 KiB table, for a 128-bit security level with 256-bit elliptic curves. Other techniques, such as the fixed-base comb method [9], can offer further storage-processing trade-offs. For example, in Section 4.3 we use the fixed comb method with $w = 8$ to run fixed-point multiplications $\approx$8 times faster than random-point multiplications, at the cost of 16 KiB. Using a smaller w and additional implementation tricks, similarly significant speed-ups can be obtained in constrained platforms using 3 KiB or less memory [15], [16]. Note that a side-channel resistant implementation is not necessary for this operation, since all values involved (h and C) are public.

Besides this performance gain, the proposed approach slightly improves the efficiency of two other processes. The first is the private key reconstruction by the user: since this key is computed simply as $u \leftarrow x + sig$, instead of $u \leftarrow h{\cdot}x + sig$ as in ECQV, it saves one modular multiplication. The second is the CA's signing procedure itself, computed as $sig \leftarrow r + h{\cdot}c$ rather than as $sig \leftarrow h{\cdot}r + c$: even though both equations involve the same number of operations, SIMPL can benefit from pre-computation techniques similar to those applicable to the public key reconstruction algorithm. After all, the modular multiplication is such that one of its factors (namely, the private key c) is usually one and the same for all users. In this case, however, the optimization approach must be side-channel resistant, because the fixed factor is private.

3.2 Security Analysis

Assuming the hardness of the elliptic curve discrete logarithm problem (ECDLP) [17], the security properties of the SIMPL scheme should be similar to those provided by ECQV. This claim can be proven under the general security model for implicit certificates described in [1]. This model, illustrated in Fig. 1, can be formulated as in Definition 1.

Definition 1 (Implicit certificate adversaries [1]). *Assume a scenario with n_{usr} legitimate users, denoted usr_i for $1 \leqslant i \leqslant n_{usr}$, and with n_{CA} CAs, denoted CA_j for $1 \leqslant j \leqslant n_{\mathrm{CA}}$. Let (X_i, j) denote usr_i's implicit certificate request for CA_j, and let $(V_i, \mathtt{meta}_i, sig_i)$ be the response sent by that CA. Also, let U_i and u_i denote, respectively, the public and the private keys reconstructed by usr_i from CA_j's response, using that CA's public key C_j. There are no restrictions on the number of certificate requests that can be sent by usr_i to CA_j.*

A (τ, ϵ)-adversary $\mathcal{A}$ of an implicit certificate scheme is a probabilistic Turing machine that runs in time at most τ, interacting with legitimate users and CAs by performing the following operations, any number of times.

- *(I) receive a request (X_i, j) from usr_i, intended for CA_j; and*
- *(II) send a request $(X'_{i'}, j')$ to $\mathrm{CA}_{j'}$, and then receive $(V'_{i'}, \mathtt{meta}'_{i'}, sig'_{i'})$ as that CA's response.*

With probability at least ϵ, $\mathcal{A}$ outputs a triple $(V, \mathtt{meta}, u)$ such that u is the private key corresponding to the public key U reconstructed from V, $\mathtt{meta}$ and some C_z — in our case, this means that $u{\cdot}G = V + \mathcal{H}(V, \mathtt{meta}, C_z) \cdot C_z$, — and either:

- *(I) [Forgery attack against CA_z]: $(V, \mathtt{meta})$ was never part of a response by CA_z; or*
- *(II) [Key compromise against usr_i]: $(V, \mathtt{meta})$ was included in a response of CA_z to some request (X_i, j) made by usr_i.*

A (τ, ϵ)-adversary $\mathcal{A}$ is considered successful if ϵ is non-negligible for a polynomial time τ.

In summary, this model covers a scenario where the attacker $\mathcal{A}$ acts as proxy for requests from users and responses from CAs. Hence, $\mathcal{A}$ can: simply relay the request to the correct CA; modify the value of X_i in the request; modify the user identifier i in the request, thus affecting the value of $\mathtt{meta}$ in the certificate; and/or forward the request to a different CA. Theorem 1 summarizes the security of the SIMPL protocol in this security model.

Theorem 1. *In the random oracle model and assuming the intractability of the ECDLP problem in $\mathbb{G}$, there is no adversary $\mathcal{A}$ that is successful against the Schnorr-based implicit certification (SIMPL) scheme in the sense of Definition 1.*

Proof. The proof is a simple adaptation of [1, Theorem 2] to a Schnorr-style signature scheme, which is known to be secure in the random oracle model assuming the intractability of the ECDLP [18]. Since this adaptation is quite straightforward, we leave the details to the Appendix , which can be found on the Computer Society Digital Library at http://doi.ieeecomputersociety.org/10.1109/TC.2020.2988637. □

3.3 Composability With Signature Schemes

Like other implicit schemes (e.g., ECQV), SIMPL is not expected to be universally composable. Therefore, using the resulting implicit keys in combination with other cryptographic primitives may require additional analysis.

In the specific context of V2X communications, though, implicitly certified keys need basically to be used for signatures. In particular, SCMS recommends the ECDSA [19] signature algorithm, although Schnorr variants may also be of interest. Fortunately, as shown in [11], Schnorr-based implicit certificates can be combined with ECDSA or Schnorr signatures in a secure fashion as long as the key-prefixing technique is employed when signing messages.

4 Application to V2X Communications

Vehicle-to-everything (V2X) communications refers to the set of technologies that allow vehicles to exchange messages among themselves and with other entities (e.g., roadside units or pedestrians) [20]. Its wide scale deployment is motivated by several factors, including improving transportation safety and efficiency. For example, if vehicles exchange Basic Safety Messages (BSM), informing their relative speed, acceleration and distance to each other, accidents can be avoided with or without the drivers' direct intervention. In addition, V2X technologies enable applications in which

Authorized licensed use limited to: Duke University. Downloaded on February 17,2026 at 16:37:47 UTC from IEEE Xplore. Restrictions apply.

TABLE 4
The Unified Butterfly Key (UBK) Certificate Provisioning Process

	Vehicle	→	RA	→	PCA	-RA→	Vehicle
(explicit)	$x \xleftarrow{\$} \mathbb{Z}_q$ $X \leftarrow x{\cdot}G$	X, f	$\hat{X}_i \leftarrow X + f(i){\cdot}G$ $(0 \leqslant i < \beta)$	$\hat{X}_i$	$r_i \xleftarrow{\$} \mathbb{Z}_q$ $U_i \leftarrow \hat{X}_i + r_i{\cdot}G$ $sig_i \leftarrow Sign(c, \{U_i, \texttt{meta}\})$ $cert_i \leftarrow \{U_i, \texttt{meta}, sig_i\}$ $pkg \leftarrow Enc(\hat{X}_i, \{cert_i, r_i\})$	pkg	$\hat{x}_i \leftarrow x + f(i)$ $\{cert_i, r_i\} \leftarrow Dec(\hat{x}_i, pkg)$ $Verif(C, cert_i)$ $u_i \leftarrow \hat{x}_i + r_i$ $u_i{\cdot}G \stackrel{?}{=} U_i$
(implicit) ECQV					$r_i \xleftarrow{\$} \mathbb{Z}_q$ $V_i \leftarrow \hat{X}_i + r_i{\cdot}G$ $cert_i \leftarrow \{V_i, \texttt{meta}\}$ $h_i \leftarrow \mathcal{H}(cert_i)$ $sig_i \leftarrow h_i \cdot r_i + c$ $pkg \leftarrow Enc(\hat{X}_i, \{cert_i, sig_i\})$		$\hat{x}_i \leftarrow x + f(i)$ $\{cert_i, sig_i\} \leftarrow Dec(\hat{x}_i, pkg)$ $h_i \leftarrow \mathcal{H}(cert_i)$ $u_i \leftarrow h_i \cdot \hat{x}_i + sig_i$ $U_i \leftarrow u_i{\cdot}G \stackrel{?}{=} h_i \cdot V_i + C$
(implicit) SIMPL					$r_i \xleftarrow{\$} \mathbb{Z}_q$ $V_i \leftarrow \hat{X}_i + r_i{\cdot}G$ $cert_i \leftarrow (V_i, \texttt{meta})$ $h_i \leftarrow \mathcal{H}(cert_i, C)$ $sig_i \leftarrow r_i + h_i \cdot c$ $pkg \leftarrow Enc(\hat{X}_i, \{cert_i, sig_i\})$		$\hat{x}_i \leftarrow x + f(i)$ $\{cert_i, sig_i\} \leftarrow Dec(\hat{x}_i, pkg)$ $h_i \leftarrow \mathcal{H}(cert_i, C)$ $u_i \leftarrow \hat{x}_i + sig_i$ $U_i \leftarrow u_i{\cdot}G \stackrel{?}{=} V_i + h_i \cdot C$

The top rows show the explicit and implicit certification models from [5]; the bottom row shows the SIMPL protocol (the gray background highlights the main differences from ECQV implicit certificates).

vehicles and drivers can be informed about adverse road conditions, such as nearby accidents, congestion, busy intersections, slippery conditions, or potholes. Such capabilities create a much more dynamic and information-rich environment than what could be achieved with regular traffic signs.

An important challenge in V2X environments is to build a Vehicular Public Key Infrastructure (VPKI) where authorized vehicles cannot be tracked, either by eavesdroppers or by the system itself [4]. One common approach for this issue is to provision vehicles with multiple, short-lived pseudonym certificates [21]. Vehicles can then avoid tracking by frequently changing the certificates employed to sign their messages while they move. The reasoning is that messages from the same and from different vehicles should be indistinguishable from each other, since they carry no identity information. Hence, messages broadcast from different locations and using distinct certificates cannot be easily linked to any given user. Nevertheless, the total number of certificates valid simultaneously should be limited (e.g., about 20 to 40 [4]). Otherwise, one vehicle might be tempted to engage in Sybil attacks: by pretending to be multiple entities, it might be able to get special treatment from congestion-aware traffic lights. Among the existing VPKI solutions, the Security Credential Management System (SCMS) [4] is one of the most relevant. Developed in cooperation with the United States Department of Transportation, SCMS's certificate issuance approach combines privacy and scalability in the so-called butterfly key expansion process. Essentially, this process enables several pseudonym certificates to be provisioned with a single request from a vehicle. At the same time, those certificates cannot be linked to the requester. The resulting architecture is quite bandwidth- and processing-efficient, even when issuing thousands of certificates to each vehicle. This is particularly true if the unified butterfly key (UBK) optimization [5] is employed, since it reduces the amount of data exchanged and also the number of operations performed during the issuance process.

In this section, we show that SCMS's efficiency can be improved via the SIMPL implicit certification approach. We also present a novel existential forgery attack, which can be mounted against butterfly keys under certain conditions; the attack builds upon the certificate misbinding issue described in Section 3, and, thus, is addressed similarly.

4.1 The Unified Butterfly Key (UBK) Expansion Process

The certificate provisioning process in SCMS involves mainly three entities:

- *Vehicle*: the entity that requests pseudonym certificates from a registration authority (RA). For better efficiency, each request leads to the provisioning of a batch containing β certificates.
- *Registration Authority (RA)*: acts as a proxy between vehicles and the Pseudonym Certificate Authority (PCA). Each request from an authorized vehicle generates β individual pseudonym certificate requests to the PCA, each one containing a different public key. To improve the vehicles' privacy, requests associated with different vehicles are shuffled together by the RA. This prevents the PCA from identifying which requests belong to the same batch, assuming there is no PCA-RA collusion.
- *Pseudonym Certificate Authority (PCA)*: responsible for issuing pseudonym certificates upon request by the RA. The public key effectively placed into the certificate is a randomized version of the key received from the RA; the PCA's response is also encrypted, so only the vehicle can retrieve its contents. This prevents the RA from identifying the owner of a certificate when it is used on the field, once again assuming there is no RA-PCA collusion.

Table 4 shows the message exchange during the UBK certificate provisioning process. All communications are made via secure channels, using standard protocols (e.g., Transport Layer Security – TLS). The top rows in this table show the explicit and implicit models described in [5]. The bottom row shows the SIMPL approach, highlighting the main differences from ECQV in gray background.

In all cases, the vehicle starts by picking a random *caterpillar* private key x and computing the corresponding *caterpillar* public key $X = x{\cdot}G$. The vehicle then sends X, together with a pseudorandom instance f, to the RA.

In response to this request, the RA expands the caterpillar public key X into β *cocoon* public keys $\hat{X}_i = X + f(i){\cdot}G$. Since f is shared only among the vehicle and the RA, the resulting cocoon keys are unlinkable to the original X from the perspective of any other entity. The RA then sends each individual $\hat{X}_i$ to the PCA, while shuffling together requests from different batches to ensure their unlinkability.

Authorized licensed use limited to: Duke University. Downloaded on February 17,2026 at 16:37:47 UTC from IEEE Xplore. Restrictions apply.

The PCA, in turn, randomizes $\hat{X}_i$ by adding $r_i \cdot G$ to it, for a randomly picked r_i. For explicit certificates, the resulting elliptic curve point is used directly as the vehicle's *butterfly* public key U_i; it is then placed into a certificate, together with any required metadata `meta` (e.g., a validity period), and signed. For implicit certificates, the randomized point is used as the butterfly reconstruction credential V_i; it is also combined with `meta`, and then signed according to the procedure described in Sections 2 and 3 (depending on the implicit model adopted). In all cases, the resulting certificate is encrypted with the originally provided $\hat{X}_i$ and sent back to the RA. The RA, unable to decrypt the PCA's response pkg, forwards it back to the requesting vehicle, in batch.

Finally, the vehicle computes the key $\hat{x}_i = x + f(i)$ for decrypting pkg. It then verifies that the retrieved certificate is indeed valid, aiming to ensure there is no Man-in-the-Middle attack by the RA [4], [5]. This is done either by checking its signature (for explicit certificates) or by performing the corresponding key verification process (for implicit ones). If the verification is successful, the keys obtained can be used for signing messages.

4.2 The risk of certificate misbinding: building (and preventing) related-key existential forgery attacks

The security of the original butterfly key expansion is discussed in [4], [20], whereas additional security aspects related to the UBK optimization are detailed in [5]. However, none of these works tackle issues arising from one inherent property of butterfly keys: the correlation among different keys from the same batch. More precisely, any pair of private keys (u_i, u_j) created from the same caterpillar key x share some well-defined relationship $\rho_{i,j} = u_i - u_j$. Indeed, for explicit certificates we have $u_i = x + f(i) + r_i$ and $u_j = x + f(j) + r_j$, meaning that $\rho_{i,j} = f(i) + r_i - f(j) - r_j$. For certificates built using SIMPL, the relationship is very similar, given by $\rho_{i,j} = f(i) + sig_i - f(j) - sig_j$. In contrast, for ECQV certificates, this relationship is more complex: since $u_i = h_i \cdot (x + f(i)) + sig_i$, we can write $\rho_{i,j} = h_i \cdot (x + f(i)) + sig_i - h_j \cdot (x + f(j)) - sig_j$.

In principle, such relationships are unknown by any entity except the owner of the keys. The reason is that $f(i)$ is a secret shared between vehicle and RA, whereas r_i and sig_i (respectively, for explicit and implicit certificates) are known only by vehicle and PCA. Hence, even though $h_i = \mathcal{H}(cert_i)$ is public, the randomness added by PCA and RA is enough to protect the correlation between any pair of keys, in all certification models.

However, it is possible to exploit such relationship among butterfly keys in at least one particular condition: (1) a collusion between PCA and RA enables them to learn the correlation between those keys; (2) a Schnorr-style signature algorithm is employed when signing data; and (3) the signatures are prone to certificate misbinding attacks analogous to the one described in Section 3. Condition 1 can be met by the explicit and by the SIMPL certification approaches in case of RA-PCA collusion, since in both cases $\rho_{i,j}$ does not depend on the value of x (known only by the vehicle). Hence, whenever Conditions 2 and 3 are met, the system becomes vulnerable to the following attack:

Definition 2 (Related-key Existential Forgery). *Suppose that two private keys u_i and u_j share some mathematical relationship that is known by the adversary, albeit the actual value of those keys remain secret. Given one or more valid signatures generated with u_i, the adversary is able to forge the signature of one (not necessarily chosen) message, as if the forgery was signed with u_j.*

To understand the attack, it is useful to recall how Schnorr digital signatures are generated with private key u and verified with the public key U. First, pick a secret random number α and compute the elliptic curve point $P = \alpha \cdot G$. To sign message m, compute the hash $\phi = \mathcal{H}(P, m)$, and then make $sig(\phi) = \alpha + \phi \cdot u$. The output of the signature process is the pair $(\phi, sig(\phi))$. The verification procedure consists basically in retrieving point $P = sig(\phi) \cdot G - \phi \cdot U$ and checking for the equality $\mathcal{H}(P, \phi) \stackrel{?}{=} \phi$.

For the butterfly key u_i, the signature $(\phi, sig(\phi)_i)$ is such that $\phi = \mathcal{H}(P, m)$ and $sig(\phi)_i = \alpha + \phi \cdot u_i)$. It turns out that, given this signature, the colluding RA and PCA can easily compute the signature for $(\phi, sig(\phi)_j)$, i.e., the signature for the same message m as if it was produced with u_j. To accomplish this, first recover $P = sig(\phi)_i \cdot G - \phi \cdot U_i$ and then simply compute $sig(\phi)_j = sig(\phi)_i - \phi \cdot \rho_{i,j}$. This is a valid signature because $sig(\phi)_j = \alpha + \phi \cdot u_j = (\alpha + \phi \cdot u_i) - \phi \cdot u_i + \phi \cdot u_j = sig(\phi)_i - \phi \cdot \rho_{i,j}$. Both signatures refer to the same vehicle, also identifiable due to the collusion, so this attack allows RA and PCA to frame a target for sending m using different identities. As a possible consequence, vehicles could be wrongly accused of performing Sybil-like attacks. Hence, the risks of an RA-PCA collusion would go beyond the system's privacy (i.e., the ability to track vehicles), but also affect its security.

Fortunately, since this issue is basically an extension of a certificate misbinding attack, it can be averted using the same technique from Section 3.3: via key prefixing. Indeed, with this approach the signature generated with u_i would be $(\phi_i, sig(\phi)_i)$, where $\phi_i = \mathcal{H}(P, \phi, h_i)$, $h_i = \mathcal{H}(cert_i)$, and $sig(\phi)_i = \alpha + \phi_i \cdot u_i)$. To employ the same trick as before, the attacker would have to compute $sig(\phi)_j = sig(\phi)_i - \phi_i \cdot u_i + \phi_j \cdot u_j = sig(\phi)_i - \phi_i \cdot \rho_{i,j} + (\phi_j - \phi_i) \cdot u_j$, which should be unfeasible without knowing u_j (or, equivalently, u_i).

At this point, it is important to emphasize that such attack does not invalidate SCMS's security claims, for at least three reasons. The first is that one of SCMS's security assumptions is that PCAs and RAs do not collude. In addition, SCMS recommends using the ECDSA signature algorithm [19], for which we were unable to reproduce the attack. Finally, the latest version of SCMS already suggests the countermeasure hereby proposed, i.e., including the signer's certificate information in the hash computation [4, Section V.D]. The motivation for this recommendation, though, was not as serious as the possibility of forgery attacks like the one hereby unveiled. All in all, to ensure that SCMS remains secure against forgery even if vehicles use a Schnorr variant for signing messages, it is sensible to mandate that all signatures made with butterfly keys include this countermeasure.

4.3 Experimental Results

In this section, we compare the performance of the explicit, ECQV-based implicit and SIMPL-based implicit certificates. For this purpose, we use an Intel i5 4570 processor and the RELIC cryptography library version 0.4.1 (https://github.com/relic-toolkit/relic), building an experimental setup similar to the one described in [5].

In all experiments, multiplications by the PCA's public key C are optimized using RELIC's fixed comb method with $w = 8$, using 16 KiB of RAM for storing a pre-computation table. We argue that this treatment of C as a fixed point, although not always advantageous, is reasonable in many real-world V2X deployments. For example, this assumption should hold when a vehicle verifies its batch of certificates, since each manufacturer is likely to adopt a single PCA [22] or a few [5] for this procedure. It should also apply to roadside units, whose implicit certificates are provided by national authorities [20], [22]. Finally, some PCAs may operate on a specific geographic region (e.g., a state) [22], so it may be the issuer of many certificates thereby encountered. In those scenarios, the vehicles' memory capabilities are likely to support the (few) look-up tables required by such optimization, in particular in state-of-the-art automotive microcontrollers with more than 700 KiB of RAM [23]. We emphasize, though, that this memory space does not need to be dedicated. After all, the look-up tables can be computed dynamically, whenever the vehicle notices the reception of many certificates signed by the same PCA.

4.3.1 Certificate Provisioning

We start by measuring the number of cycles on all entities involved in the SCMS certificate provisioning processes, using the recommended

Authorized licensed use limited to: Duke University. Downloaded on February 17,2026 at 16:37:47 UTC from IEEE Xplore. Restrictions apply.

TABLE 5
Processing Costs (in Cycles) for the UBK Approach When Issuing β Certificates With the Explicit, ECQV, and SIMPL Approaches

	Vehicle	→	RA	→	PCA	-RA→	Vehicle
UBK explicit	254×10^3	-	$\beta \cdot (250 \times 10^3)$	-	$\beta \cdot (2.86 \times 10^6)$	-	$\beta \cdot (2.73 \times 10^6)$
UBK implicit (ECQV)					$\beta \cdot (2.47 \times 10^6)$		$\beta \cdot (4.19 \times 10^6)$
UBK implicit (SIMPL)					$\beta \cdot (2.47 \times 10^6)$		$\beta \cdot (2.45 \times 10^6)$
Ratio: SIMPL/explicit	1	-	1	-	0.86	-	0.91
Ratio: SIMPL/ECQV					1		0.58

algorithms [20]: ECDSA [19] for signatures and ECIES [9] for asymmetric encryption/decryption, both configured with a 128-bit security level. We do so mostly for completeness, though, since the most relevant performance gains are on the vehicles' side, when they verify their public keys. Table 5 depicts the results obtained, considering an average of 1000 executions of each operation and a standard deviation below 1 percent. As shown in this table, the processing cost of the proposed scheme is reasonably lower at the vehicle verifying its own certificate batch. Namely, the costs of the proposed implicit approach is 91 and 58 percent of the cost taken, respectively, by UBK in the explicit and in the ECQV implicit model.

Finally, it is interesting to notice that the adoption of implicit certificates in SCMS seems to have been motivated for the two reasons highlighted in [2, page 17]: (1) it potentially leads to smaller certificates and, hence, to lower bandwidth usage in V2X communications; and (2) during operation, the process of verifying a message's signature can be combined with public key verification for better computational efficiency. Nevertheless, when compared to explicit certificates, the ECQV leads to a more costly certificate verification procedure: as shown in Table 5, the processing costs at the vehicle's side with ECQV implicit certificates is roughly 1.53× the one obtained in the explicit model. This potential source of criticism is overcame by the SIMPL scheme, though, since in this case the implicit approach is actually less costly than the explicit counterpart. Therefore, SIMPL allows SCMS to fully benefit from the implicit certification model's bandwidth savings.

4.3.2 Operation: BSM Verification

We also evaluate the benefits of SIMPL when vehicles verify implicit certificates from surrounding peers, and then the corresponding ECDSA signatures on BSMs. The gains in this case come from the following observations.

Ignoring ancillary operations (e.g., hashing), the cost of verifying a ECDSA signatures consists basically of two elliptic curve scalar multiplications: one by the generator G, which can be implemented as a fixed point at cost μ_f; and another by the signer's public key U, which is usually handled as a random point at cost $\mu_r > \mu_f$. This latter multiplication can also be performed indirectly. More precisely, the result of $z \cdot U$ for some scalar z can be obtained by computing $z \cdot h \cdot V + z \cdot C$ (if ECQV certificates are used [2]) or $z \cdot V + z \cdot h \cdot C$ (in the case of SIMPL certificates). In either case, and without any additional tricks, the trade-off for not computing U directly is $(\mu_f + \mu_r)$, due to the multiplications by the random point V and by the (potentially) fixed-point C. Consequently, ignoring point additions, each BSM verification would take basically: μ_f, for the multiplication by G; plus $(\mu_f + \mu_r)$, for the indirect computation of $z \cdot U$. Verifying n individual signatures under the same U in this setting takes, thus, $n \cdot (2\mu_f + \mu_r)$.

On the other hand, computing U in SIMPL takes roughly μ_f. Hence, it is always beneficial to perform such computation beforehand, rather than doing it indirectly. After all, in this case the cost of verifying n signatures made with U would be around $\mu_f + n \cdot (\mu_f + \mu_r)$ instead of $n \cdot (2\mu_f + \mu_r)$. This optimization contrasts with ECQV-based implicit certificates, for which computing U directly is not much advantageous in general. The reason is that the cost of verifying BSM signatures from the same sender would then be roughly $\mu_r + n \cdot (\mu_f + \mu_r)$, which is larger than $n \cdot (2\mu_f + \mu_r)$ for small values of n. Indeed, as shown in Fig. 2, the verification of BSMs signed with SIMPL-based keys (curve III) is 1.5 to 8.7 percent faster than with ECQV (curve II) if the vehicle computes U before performing 1 to 50 BSM verifications. Furthermore, the advantage of both ECQV and SIMPL over an approach where U is not computed directly (curve I) grows with the number of BSMs signed under U.

Such costs may be influenced by many implementation tricks, like multi-point multiplication [9] or batch verification [24]. Nonetheless, they give a glimpse of SIMPL's potential to speed up signature verification in V2X environments.

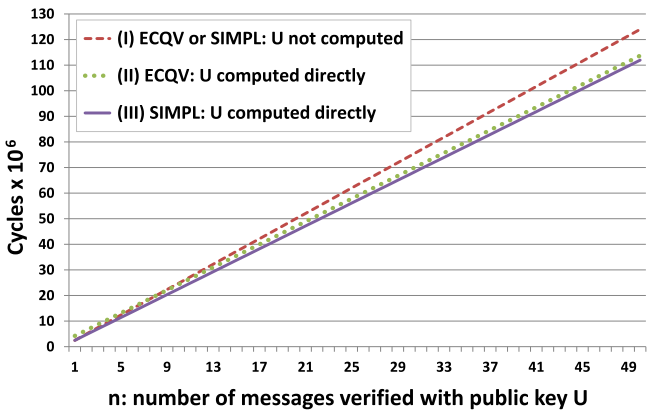


Fig. 2. Processing costs (in cycles) for verifying n BSMs with the same public key U, using ECQV- and SIMPL-based implicit certificates. When U is computed directly, such computation is included in the costs.

5 Conclusion

The main benefit of the implicit certification model is that it leads to smaller certificate material than traditional approaches. In many cases, however, this comes at the cost of increased processing when verifying the validity of implicitly certified public keys.

In this article, we show discuss an implicit certification scheme that tackles the main bottleneck of implicit certification schemes such as [2]: the costly scalar multiplication by EC points involved in the public key verification process. Specifically, by using an OMC-like, Schnorr-based signature approach for certifying keys, random-point EC multiplications can be replaced by fixed-point operations. The resulting scheme supports, thus, optimized implementations that in principle cannot be applied to the commonly used ECQV implicit certificates. It is also compliant with the SCMS VPKI [4] in terms of message formats and underlying signature algorithms, requiring only a small modification on how the corresponding implicit certificates are generated by PCAs and validated by vehicles.

We also evaluate the actual gains of the proposed implicit certification approach in the V2X context. Namely, we benchmark an implementation of SCMS with the UBK optimization [5] considering the explicit model, the ECQV scheme, and the proposed SIMPL

Authorized licensed use limited to: Duke University. Downloaded on February 17,2026 at 16:37:47 UTC from IEEE Xplore. Restrictions apply.

implicit certification. Our results show that SIMPL enables performance gains during certificate provisioning and usage, while preserving the bandwidth savings typical of implicit certificates.

As an additional contribution, we present a novel existential forgery attack that can be mounted against butterfly keys under certain conditions. The attack exploits the underlying properties of butterfly keys, namely, the correlation between pairs of keys from the same vehicle. Therefore, this vulnerability applies not only to the original SCMS [20], but also to the UBK optimization proposed in [5]. We also show a simple fix for this issue, which was already suggested in the most recent SCMS specification from [4], although for different reasons.

Acknowledgments

This work was supported by LG Electronics, and by the Brazilian CAPES (Fin. Code 001), CNPq (Grant 301198/2017-9) and FAPESP (Grant 13/25977-7). The authors would like to thank Zhaohui Cheng and Paul Bottinelli for their insightful comments on an early version of this manuscript.

References

[1] D. Brown, R. Gallant, and S. Vanstone, "Provably secure implicit certificate schemes," in *Proc. 5th Int. Conf. Financial Cryptography*, 2002, pp. 156–165.
[2] Certicom, "SEC 4: Elliptic curve Qu-Vanstone implicit certificate scheme (ECQV)," Certicom Research, Tech. Rep., 2013, [Online]. Available:www.secg.org/sec4-1.0.pdf.
[3] M. Simplicio, M. Silva, R. Alves, and T. Shibata, "Lightweight and escrow-less authenticated key agreement for the Internet of Things," *Comput. Commun.*, vol. 98, pp. 43–51, 2017.
[4] B. Brecht *et al.*, "A security credential management system for V2X communications," *IEEE Trans. Intell. Trans. Syst.*, vol. 19, no. 12, pp. 3850–3871, Dec. 2018.
[5] M. Simplicio, E. Cominetti, H. Kupwade-Patil, J. Ricardini, and M. Silva, "The unified butterfly effect: Efficient security credential management system for vehicular communications," in *Proc. IEEE Veh. Netw. Conf.*, 2018, pp. 1–8.
[6] T. Weil, "VPKI hits the highway –- secure communication for the US DOT connected vehicle pilot program," May 2017. [Online]. Available: https://www.securityfeeds.us/sites/default/files/VPKI_Hits_Highway_DineLearn_9May2017.pdf
[7] L. Pintsov and S. Vanstone, "Postal revenue collection in the digital age," in *Financial Cryptography*. Berlin, Germany: Springer, 2001, pp. 105–120.
[8] D. Brown, M. Campagna, and S. Vanstone, "Security of ECQV-certified ECDSA against passive adversaries," Cryptology ePrint Archive, Report 2009/620, 2009, [Online]. Available: https://eprint.iacr.org/2009/620.
[9] D. Hankerson, A. Menezes, and S. Vanstone, *Guide to Elliptic Curve Cryptography*. New York, NY, USA: Springer, 2003.
[10] C. Schnorr,"Efficient signature generation by smart cards," *J. Cryptol.*, vol. 4, no. 3, pp. 161–174, 1991.
[11] Z. Cheng and L. Chen, "Certificateless public key signature schemes from standard algorithms," in *Information Security Practice and Experience*. Berlin, Germany: Springer, 2018, pp. 179–197.
[12] H. Krawczyk, "SIGMA: The 'SIGn-and-MAc' approach to authenticated Diffie-Hellman and its use in the IKE protocols," in *Proc. Int. Conf. Cryptol.*, 2003, pp. 400–425.
[13] D. Bernstein, "Multi-user schnorr security, revisited," Cryptology ePrint Archive 2015/996, 2015, [Online]. Available:https://eprint.iacr.org/2015/996
[14] R. Canetti and H. Krawczyk, "Analysis of key-exchange protocols and their use for building secure channels," in *Proc. Int. Conf. Cryptol.*, 2001, pp. 453–474.
[15] Z. Liu, H. Seo, J. Großschädl, and H. Kim, "Efficient implementation of NIST-compliant elliptic curve cryptography for 8-bit AVR-based sensor nodes," *IEEE Trans. Inf. Forensics Security*, vol. 11, no. 7, pp. 1385–1397, Jul. 2016.
[16] H. Fujii and D. Aranha, "Curve25519 for the Cortex-M4 and beyond," in *Proc. LatinCrypt*, 2019, pp. 109–127.
[17] K. E. Lauter and K. E. Stange, "The elliptic curve discrete logarithm problem and equivalent hard problems for elliptic divisibility sequences," in *Proc. SAC*, 2008, pp. 309–327.
[18] Y. Seurin, "On the exact security of Schnorr-type signatures in the random oracle model," in *Proc. 31st Annu. Int. Conf. Theory Appl. Cryptogr. Techn.*, 2012, pp. 554–571.
[19] NIST, *FIPS 186–4: Digital Signature Standard (DSS)*, National Institute of Standards and Technology, Jul. 2013. [Online]. Available: http://nvlpubs.nist.gov/nistpubs/FIPS/NIST.FIPS.186–4.pdf
[20] CAMP, "Security credential management system proof–of–concept implementation – EE requirements and specifications supporting SCMS software release 1.1," Vehicle Safety Communications Consortium, Tech. Rep. DTNH22-14-H-00449/0003, May 2016.
[21] J. Petit, F. Schaub, M. Feiri, and F. Kargl, "Pseudonym schemes in vehicular networks: A survey," *IEEE Commun. Surv. Tuts.*, vol. 17, no. 1, pp. 228–255, Firstquarter 2015.
[22] USDOT, "National security credential management system (SCMS) deployment support – SCMS baseline summary report," US Department of Transportation (USDOT), Tech. Rep., Jan. 2018.
[23] L. Popa, B. Groza, and P. Murvay, "Performance evaluation of elliptic curve libraries on automotive-grade microcontrollers," in *Proc. 14th Int. Conf. Availability Rel. Secur.*, 2019, pp. 100:1–100:7.
[24] P. Bottinelli and R. Lambert, "Accelerating V2X cryptography through batch operations," Cryptology ePrint Archive, Report 2019/887, 2019, [Online]. Available: https://eprint.iacr.org/2019/887.

▷ **For more information on this or any other computing topic, please visit our Digital Library at** www.computer.org/csdl.

Authorized licensed use limited to: Duke University. Downloaded on February 17,2026 at 16:37:47 UTC from IEEE Xplore. Restrictions apply.